

RELATÓRIO DO PROJETO

Relatório Técnico: Pipeline Híbrido de Modernização SQL para Python

Ambiente: Python 3.14 · PostgreSQL 16 ·
LangGraph

Orquestração: Multi-LLM (Gemini / OpenAI /
OpenRouter)

Data de Emissão: Outubro de
2026

1. Resumo Executivo

Este documento consolida a arquitetura, metodologia e resultados práticos do **Pipeline Híbrido de Modernização SQL para Python**, desenvolvido para converter procedimentos e funções legadas em PL/pgSQL para código idiomático e auditável em Python 3.14.

- O Desafio:** A modernização de rotinas legadas em PL/pgSQL para linguagens de uso geral envolve riscos críticos para as operações de negócio. O código legado entrelaça controle transacional explícito (`FOR UPDATE`, savepoints), cursores imperativos, cálculos financeiros sensíveis (`NUMERIC(18,2)`) e manipulações diretas no banco de dados. Tentar uma reescrita puramente manual gera prazos imprevisíveis e alto índice de regressão; por outro lado, confiar cegamente em modelos de linguagem (LLMs) sem salvaguardas gera alucinações de sintaxe, tipos incorretos e violações de integridade transacional.
- A Abordagem Pragmática:** A arquitetura não busca criar um compilador perfeito de PL/pgSQL (uma tarefa de complexidade acadêmica que inviabilizaria a entrega), mas sim uma **esteira de produção controlada**. A premissa central é isolar a geração estocástica da LLM atrás de barreiras rígidas de validação sintática estática e testes comportamentais automatizados em sandbox. Nenhuma saída é aceita ou promovida sem passar por critérios objetivos e verificáveis.
- Resultados Principais:**
 - Validação comportamental bem-sucedida com **100% de paridade** em cenários reais para rotinas de negócio do domínio bancário, notadamente a função `fn_saldo_cliente` (Anexo B) e a procedure `sp_atualizar_status_contas_inativas` (Anexo C), com isolamento de schema temporário e comparação determinística de estado.
 - Implementação de um motor de avaliação no endpoint `/evaluation` com cálculo de métricas rigorosas e auditáveis, assegurando transparência absoluta sobre o status de cada artefato processado.

2. Desenho Arquitetural e Decisões Técnicas

A arquitetura do pipeline reflete uma disciplina de engenharia focada em previsibilidade, desacoplamento e contenção de custos operacionais.

- Monólito Modular (FastAPI + LangGraph):**

Optou-se por uma aplicação unificada que combina FastAPI para a exposição da API REST e LangGraph para a orquestração do fluxo em grafo acíclico dirigido (DAG). Essa escolha minimizou a latência de rede entre etapas e eliminou a sobrecarga de gerenciar múltiplos microsserviços distribuídos durante a conversão. Ao mesmo tempo, garantiram-se fronteiras de domínio rígidas através de módulos estritamente delimitados:

- `pipeline.parser`: Responsável pela inspeção estrutural determinística;

- `pipeline.llm`: Adaptador isolado de inferência com prompts versionados;
- `pipeline.validation`: Análise estática de código (AST e Ruff);
- `pipeline.db`: Camada de persistência e auditoria de estado em PostgreSQL.

O fluxo opera como uma máquina de estados finita onde cada nó recebe e devolve um estado tipado (`PipelineState`).

- **Agnosticismo de LLM (Adaptador Multi-Provedor):**

A camada de inferência foi projetada para ser completamente agnóstica em relação ao provedor de IA. O sistema suporta nativamente o **Google Gemini** (via SDK oficial `google-genai`), além de **OpenAI** e instâncias distribuídas via **OpenRouter** (através do cliente com padrão de API aberta). A seleção pode ser realizada dinamicamente a nível de requisição (`provider`, `model_name`, `api_key`), permitindo trocar modelos proprietários por modelos abertos ou corporativos sem alterar uma única linha da lógica do orquestrador LangGraph.

- **O Padrão *Fail-Fast* no Orquestrador:**

Ciclos infinitos de correção ("self-healing loops") são conhecidos por inflacionar custos de tokens e induzir a LLM a alucinações sucessivas em cascata. Para conter esse risco, o pipeline adota uma política restrita de **no máximo 1 tentativa de reparo** em caso de falha de sintaxe ou lint. Se a geração inicial falhar na validação estática, o erro detalhado da AST ou do linter é injetado em um prompt de reparo. Caso a segunda tentativa permaneça inválida, o nó de validação encaminha a execução imediatamente para a finalização com status `failure` e os nós subsequentes são marcados como `stage_skipped`. Prioriza-se a contenção de custos e a transparência em relação à insistência estocástica descontrolada.

3. Extração Estrutural vs. Árvore Sintática (AST)

A análise sintática de linguagens procedurais de banco de dados representa um dos maiores gargalos da engenharia de modernização.

- **Representação Intermediária (IR) Pragmática:**

Compiladores formais de SQL (como SQLGlott ou pglast) são altamente eficazes em comandos DML e DDL padrão, mas falham sistematicamente ao processar blocos imperativos complexos de PL/pgSQL (`DECLARE ... BEGIN ... EXCEPTION ... END`, cursores e declarações de tipos dinâmicos). Em vez de tentar construir uma gramática BNF completa do dialeto procedural do PostgreSQL, a arquitetura adotou um **parser híbrido de extração estrutural**. Este parser extrai de forma determinística a assinatura da rotina (nome, parâmetros de entrada/saída, tipos de retorno, modo de linguagem) e constrói uma **Representação Intermediária (IR)** tipada. O corpo procedural complexo é então empacotado com contexto rico e metadados estruturados, delegando a transposição lógica para a inferência probabilística da LLM.

- **Gestão de Risco e Construções Desconhecidas:**

Estruturas sintáticas não reconhecidas pelo scanner não provocam travamentos abruptos (*unhandled crashes*). Em vez disso, são detectadas e registradas no campo `unknown_constructs` da IR. Essa lista de construções desconhecidas é incluída no relatório final da execução como limitações mapeadas, alertando os engenheiros de revisão sobre quais pontos demandam inspeção manual obrigatória.

4. Metodologia de Garantia de Qualidade e Avaliação

A confiabilidade do código modernizado é garantida por uma esteira de verificação em dois níveis complementares.

- **Validação em Duas Etapas:**

1. *Etapa Estática (Gramatical e Estilo)*: Todo código gerado pela LLM passa imediatamente pelo módulo `ast.parse` do Python (garantindo que o código é sintaticamente válido em Python 3.14) e pelo linter **Ruff** (assegurando aderência às boas práticas PEP, ausência de variáveis indefinidas e imports corretos). Código que não compila é rejeitado antes de qualquer interação externa.

2. *Etapa Comportamental (Semântica de Negócio)*: A aprovação sintática não é tratada como atestado de equivalência funcional. O código aprovado estaticamente é submetido ao harness de avaliação comportamental, que confronta a execução do PL/pgSQL legado contra o código Python gerado sob as mesmas condições de entrada.

- **Isolamento Comportamental (Sandbox Determinístico):**

Para avaliar funções e procedures que realizam mutação de dados (como atualizações de saldo e transferências), foi construído um framework de testes em sandbox isolado:

- A execução cria um schema temporário descartável no PostgreSQL (`CREATE SCHEMA ...`);
- Aplica o DDL do Anexo A e efetua o *seeding* determinístico de cenários de teste;
- Executa a rotina legada em um ambiente e a função modernizada em Python em outro;
- Realiza a **normalização de campos dinâmicos**, mascarando ou abstraindo colunas voláteis como *timestamps* de auditoria (`criado_em` , `NOW()`) e geradores seriais de IDs;
- Compara o estado das tabelas e o valor de retorno linha a linha, garantindo paridade funcional sem poluir nem degradar bases de homologação ou produção.

- **O "Denominador Honesto" no Endpoint `/evaluation` :**

Na maioria dos relatórios de engenharia assistida por IA, métricas de assertividade sofrem de viés de seleção (*cherry-picking*), desconsiderando chamadas que deram timeout, erros de chave ou falhas sintáticas prévias. O endpoint `/evaluation` deste projeto implementa o conceito de **denominador honesto**: a taxa de sucesso é calculada sobre o **total absoluto de execuções submetidas**, incluindo falhas de parsing, quedas de validação, execuções simuladas (stubs) e erros de rede. Essa métrica oferece à liderança técnica uma visão fidedigna e não inflacionada da maturidade do pipeline.

5. Rastreabilidade e Gestão de Estado

Para suportar auditorias corporativas e conformidade regulatória bancária, o ciclo de vida de cada requisição é persistido de forma resiliente.

- **Persistência Preemptiva:**

Assim que o endpoint `/modernize` recebe uma carga de trabalho, ele imediatamente gera um identificador único (`run_id`) e realiza a persistência preemptiva do registro no PostgreSQL (tabela `modernization_history`), com status inicial `pending` e metadados em formato `JSONB` . Dessa forma, mesmo que ocorra uma falha catastrófica de infraestrutura, encerramento de container, timeout de rede ou estouro de cota da LLM (Erro HTTP 429), a execução nunca se torna um "trabalho fantasma". O rastro fica gravado e pode ser inspecionado posteriormente.

- **Histórico Granular de Execução:**

A tabela de histórico preserva a íntegra dos dados da migração:

- O código-fonte SQL original submetido;

- O hash criptográfico do código (para detecção de duplicidades);
 - O estado completo de cada nó do LangGraph (`parsing` , `semantic_analysis` , `generation` , `validation` , `repair` , `finalization`);
 - O código Python gerado em cada iteração;
 - O relatório de conformidade, incluindo tempo de execução por etapa, alertas e diagnósticos do linter.
-

6. Limitações Conhecidas e Trade-offs

A transparência quanto aos limites da solução é fundamental para a governança e planejamento das fases subsequentes.

- **Dívida Técnica Explícita no Runtime (`langgraph-api==0.10.3`):**

O orquestrador LangGraph CLI utiliza a dependência `langgraph-api` fixada na versão `0.10.3` , classificada como *End of Life* (EOL) pela biblioteca mantenedora. Esta restrição foi assumida conscientemente para garantir compatibilidade com as rotas customizadas de injeção da API FastAPI sem quebrar o roteamento local. A atualização para versões mais recentes exigirá refatoração do ponto de entrada do servidor de desenvolvimento.

- **Limitações de Transação e Ausência de *Crash Recovery* Automático:**

Caso o processo do servidor sofra um encerramento forçado (*SIGKILL*) durante a inferência da LLM, o registro no PostgreSQL permanecerá com o status `pending` . Não há, na versão atual, um worker em segundo plano ou cronjob de *dead letter queue* (DLQ) para reprocessar ou marcar automaticamente como `abandoned` requisições órfãs.

- **Proibição de Otimização Autônoma (Preservação de Bugs Legados):**

Uma premissa inegociável da engenharia de migração é que a LLM **não tem autorização para "corrigir" bugs de negócio legados de forma autônoma**. Se uma procedure original omitir um arredondamento ou tiver uma condição de borda inconsistente, o código Python correspondente deve reproduzir fielmente esse comportamento na fase inicial. Alterações de regra de negócio sem alinhamento prévio rompem a equivalência funcional com os sistemas clientes existentes e mascaram diferenças contábeis.

7. Roadmap e Próximos Passos

O caminho de evolução do pipeline estabelece marcos claros para transição do ambiente de laboratório para a operação corporativa.

- **Observabilidade Avançada com Langfuse:**

A infraestrutura do pipeline já possui o conector desacoplado para integração com o **Langfuse** (utilizando o SDK v4 e traces compatíveis com OpenTelemetry). O próximo passo é a ativação em larga escala das credenciais do coletor corporativo, habilitando o monitoramento de latência por nó do grafo, telemetria detalhada de consumo de tokens por provedor/modelo e rastreamento de custos por rotina migrada.

- ***Shadow Deployment* (Execução em Modo Sombra):**

Para a homologação final do código Python gerado em ambiente produtivo, planeja-se a implantação de um mecanismo de *Shadow Deployment*. Nessa arquitetura, chamadas a stored procedures legadas no PostgreSQL são espelhadas assincronamente para a nova API em Python:

- O sistema legado responde à transação do usuário;

- O serviço Python executa os mesmos parâmetros em paralelo em um contexto de leitura ou shadow database;
 - As saídas, tempos de execução e efeitos colaterais são comparados continuamente por telemetria;
 - Qualquer divergência semântica é catalogada antes da virada definitiva da chave (*cutover*).
-

Relatório técnico emitido automaticamente pelo Pipeline Híbrido de Modernização SQL para Python.